

FORMAL LANGUAGES AND COMPILERS

prof Luca Breveglieri

Exam of Wed 5 September 2017 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

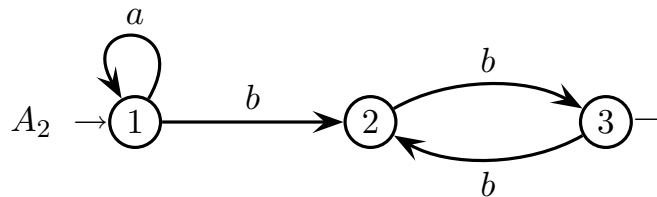
1. Consider a two-letter alphabet $\Sigma = \{ a, b \}$ and answer the following questions:

(a) Consider the following regular expression R_1 , over the alphabet Σ :

$$R_1 = (a a)^* b^+$$

By using the Berry-Sethi (BS) method, find a deterministic automaton A_1 equivalent to the regular expression R_1 . If necessary minimize the automaton A_1 , or reasonably justify that it is already minimal.

(b) Consider the following automaton A_2 , over the alphabet Σ :



Find a (possibly non-deterministic) automaton A_3 that recognizes the union language $L(A_1) \cup L(A_2)$. This automaton A_3 may have ε -transitions (spontaneous transitions), and must have only one initial state and only one final state. Then eliminate all the ε -transitions and obtain a (possibly non-deterministic) automaton, still with only one initial state and only one final state.

- (c) By using the node elimination (Brzozowsky-McCluskey - *BMC*) method, find a regular expression R_3 equivalent to the automaton A_3 .
- (d) (optional) Determine if the regular expression R_3 obtained at point (c) is ambiguous or not, and reasonably justify your answer. In case of ambiguity, find an equivalent yet unambiguous regular expression R_4 and reasonably justify it.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider a two-letter alphabet $\Sigma = \{ a, b \}$ and answer the following questions:

(a) Define a *BNF* grammar G_1 that generates the following language L_1 :

$$L_1 = \{ a^i b^j \mid i, j \geq 0 \text{ and } i \neq j \}$$

(b) Draw a syntax tree of the grammar G_1 for the valid string $a a a b \in L_1$.

(c) Say if grammar G_1 is ambiguous or not, and adequately justify your answer. In case it is ambiguous, write an equivalent yet unambiguous *BNF* grammar G'_1 .

(d) (optional) Define a *BNF* grammar G_2 that generates the following language L_2 :

$$L_2 = \{ a^i b^j c^k \mid i, j, k \geq 0 \text{ and } (i \neq j \text{ or } j \neq k) \}$$

Notice that the alphabet of language L_2 has three letters: a , b and c .

2. Consider a programming language the phrases of which are set-theoretic expressions, defining sets of numbers. Below are the composition rules of such expressions:

- An element of a set is an integer in base 10, greater than or equal to zero, unsigned and without leading digits 0.
- A variable of type set is an alphabetical identifier of this type: it may be any sequence (of length ≥ 1) of uppercase letters and underscore signs ‘_’, it must begin and terminate by a letter, and it may not contain adjacent underscores.
- A set is a list, enclosed by graph parentheses ‘{’ and ‘}’, of objects of these four types:
 - elements
 - variables
 - sets
 - expressions

in any arrangement, separated by commas ‘,’; the empty set is also allowed.

- An expression may contain the following operators, all applicable to variables, sets and subexpressions: union denoted as ‘ $\cup$ ’ and intersection denoted as ‘ $\cap$ ’, both written in the infix position; and complement denoted as ‘ $\neg$ ’, written in the prefix position.
- Both the union and the intersection operators are left-associative.
- The syntax does not specify any precedence order between the union and intersection operators, and they are simply computed from left to right, but the complement operator takes precedence over both union and intersection.
- A (sub)expression may contain subexpressions, delimited by the round parentheses ‘(’ and ‘)’, at any depth level.
- A language phrase is a single set-theoretic expression, as defined above.

Further detailed information about the lexical and syntactic structure of the programming language can be inferred from the list of six sample expressions below (each of them is a single language phrase):

$$\{ \} \tag{1}$$

$$\{ 1, 20 \} \tag{2}$$

$$\{ A, 23, A \cup B \cup SET \} \tag{3}$$

$$\{ B \cap A, 32, \{ 3, \neg C \}, \{ \}, ZZZ \} \tag{4}$$

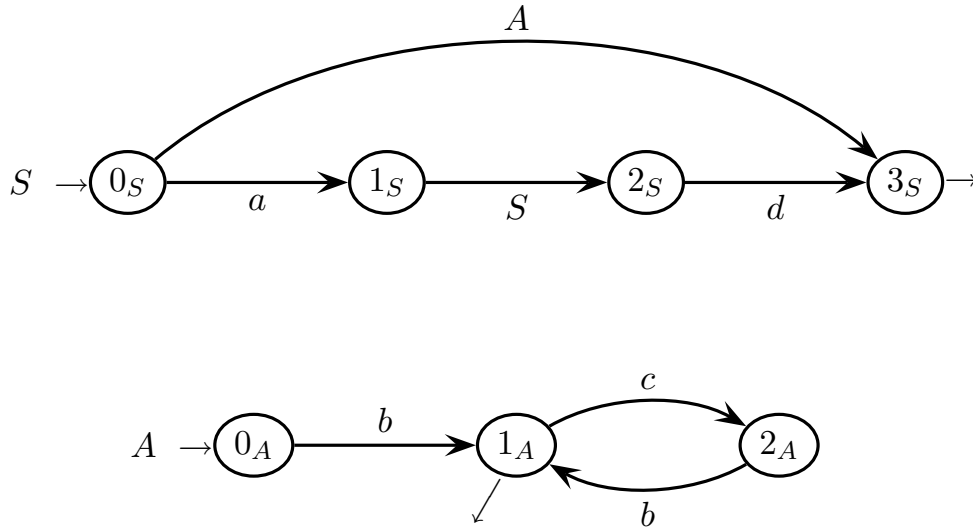
$$A \cup SET_A \cap \neg C \cup \neg \{ A \cup B, 45 \} \tag{5}$$

$$\{ 6708, \neg (A \cup B) \} \tag{6}$$

Write a grammar, not ambiguous, that models this kind of set-theoretic expression. Test your grammar and draw the syntax tree of the sample phrase (5).

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$ and the two-letter nonterminal alphabet $V = \{ S, A \}$ (axiom S):



Answer the following questions (for each of them, space is prepared on the next pages):

- (a) Draw the complete pilot of grammar G , show that grammar G is of type $ELR(1)$ and shortly justify your answer.
- (b) Simulate the ELR analysis of the following valid sample string:

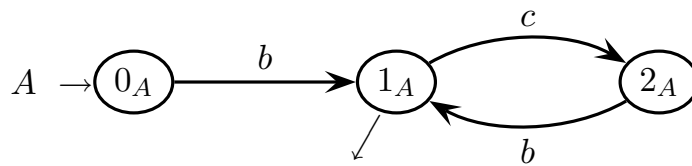
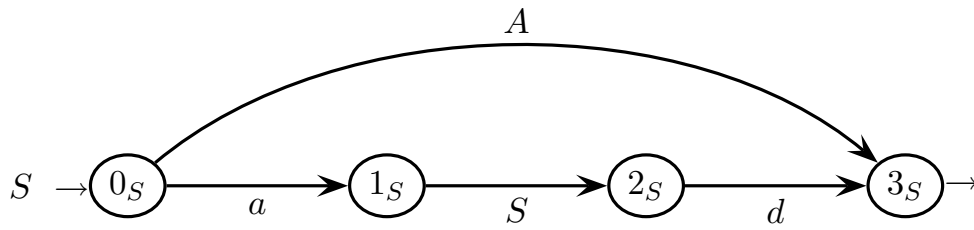
$$a b c b d \in L(G)$$

Please use the table prepared on the next page.

- (c) Find all the guide sets on the arcs of the machine net of grammar G (use the net on the next page) and argue that grammar G is of type $ELL(1)$.
- (d) (optional) Write the syntactic procedures of nonterminals S and A for the recursive descent syntax analyzer of grammar G .

here draw the pilot of grammar G
and show that grammar G is of type $ELR(1)$ - question (a)

here complete the net of grammar G with the call arcs,
 write the guide sets on all the arcs
 and argue that the grammar is of type $ELL(1)$ - question (c)



here write the syntactic procedures of nonterminals S and A - question (d)

4 Language Translation and Semantic Analysis 20%

1. Consider a three-letter terminal alphabet $\Sigma = \{0, 1, c\}$ and the following source grammar G , of type *BNF* (axiom X):

$$G \left\{ \begin{array}{l} X \rightarrow 0 X 0 \\ X \rightarrow 1 X 1 \\ X \rightarrow c \end{array} \right.$$

The source grammar G generates the palindromes of bits with centre c .

Answer the following questions:

- (a) Consider the syntactic translation τ_1 of the source language $L(G)$ that outputs the palindromes and strips off the longest suffix of bits 0 (if any). Examples:

$$\begin{aligned} \tau_1(c) &= c & \tau_1(1c1) &= 1c1 & \tau_1(00c00) &= 00c \\ \tau_1(01c10) &= 01c1 & \tau_1(0010c0100) &= 0010c01 \end{aligned}$$

Write a *BNF* translation grammar (or scheme) G_1 for translation τ_1 . If necessary, change the source grammar. Test grammar G_1 by drawing the (source and destination) trees of the sample $\tau_1(01c10) = 01c1$.

- (b) Consider the syntactic translation τ_2 of the source language $L(G)$ that outputs the palindromes and strips off the longest inner substring (if any) of bits 0 occurring immediately on the right of the centre c . One example suffices:

$$\tau_2(0100c0010) = 0100c10$$

Write a *BNF* translation grammar (or scheme) G_2 for translation τ_2 . If necessary, change the source grammar. Test grammar G_2 by drawing the (source and destination) trees of the sample $\tau_2(0100c0010) = 0100c10$.

- (c) (optional) Say if the translation grammar (or scheme) G_1 written before is deterministic or not, and justify your answer. Say if the translation τ_2 defined before is deterministic or not (independently of the grammar G_2 written for it), and justify your answer.

2. Consider the expressions defined by the following abstract syntax (axiom S):

$$\begin{cases} 1: S \rightarrow a & \text{--- } a \text{ is an atom (constant or variable)} \\ 2: S \rightarrow S \text{ op } S & \text{--- } op \text{ is a generic binary operator} \\ 3: S \rightarrow ' (' S ') ' & \text{--- parenthesized subexpression} \end{cases}$$

We want to verify the following two constraints that an expression should satisfy:

- in total, an expression may not contain more than five atoms
- in any position, an expression may not contain more than two pairs of immediately nested parentheses

Examples of semantically correct and incorrect expressions:

<i>expression</i>	<i>correct</i>	<i>reason</i>
$a \text{ op } (a \text{ op } a \text{ op } a)$	yes	
$a \text{ op } (a \text{ op } ((a \text{ op } a)))$	yes	
$(a \text{ op } a \text{ op } a) \text{ op } (a \text{ op } (a \text{ op } a))$	no	has $6 > 5$ atoms
$(a \text{ op } (a \text{ op } a)) \text{ op } (((a \text{ op } (a \text{ op } a))))$	no	has $6 > 5$ atoms and has 3 pairs of immediately nested parentheses

Attributes permitted to use (no other attributes are allowed):

<i>name</i>	<i>type</i>	<i>domain</i>	<i>meaning</i>
<i>min</i>	left	integer	number of pairs of immediately nested parentheses in the (sub)tree rooted at S
<i>na</i>	left	integer	number of atoms in the (sub)tree rooted at S
<i>OK</i>	left	boolean	true if the (sub)expression rooted at S satisfies both the constraints listed above, false otherwise

Answer the following questions:

- Write an attribute grammar (in the table prepared on the next page), based on the above syntax and using only the three attributes listed above, that computes the value of the attribute *OK* in the tree root.
- Decorate with the attribute values the sample syntax tree (prepared on the next page) of the syntactically valid yet semantically *incorrect* expression below:

$$(a \text{ op } (a \text{ op } a)) \text{ op } (((a \text{ op } (a \text{ op } a))))$$

which is the last (fourth) sample expression listed above.

Attributes permitted to use (no other attributes are allowed):

<i>name</i>	<i>type</i>	<i>domain</i>	<i>meaning</i>
<i>min</i>	left	integer	number of pairs of immediately nested parentheses in the (sub)tree rooted at S
<i>na</i>	left	integer	number of atoms in the (sub)tree rooted at S
<i>OK</i>	left	boolean	true if the (sub)expression rooted at S satisfies both the constraints listed above, false otherwise

attribute grammar to write - question (a)

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1: $S_0 \rightarrow a$

2: $S_0 \rightarrow S_1 \text{ op } S_2$

3: $S_0 \rightarrow '(S_1)'$

syntax tree of the (semantically incorrect) expression

$$(a \text{ op } (a \text{ op } a)) \text{ op } ((a \text{ op } (a \text{ op } a)))$$

to be decorated with all the attribute values - question (b)

